

Developer Manual

Digitex School Management System

Version: Laravel 11 | **Document date:** June 22, 2026

E-Digitex SMS — Multi-Institution Education Platform

Confidential — For authorized users only

Digitex SMS — Developer Manual

Module-by-Module Technical Reference

Audience: Developers, integrators, and technical school IT staff.

Companion: User Manual (layman flows), REST API Manual (hardware/mobile endpoints).

Example tenant used in examples:

- Institution ID: 3
- Name: Green Valley International School (GVIS)
- Session ID: 12 (2025-2026)
- Class Section ID: 45 (Grade 5A)
- Student ID: 1001 (Marie Kouassi)

How This Manual Is Organized

Each module section includes:

Section	Content
Purpose	Business reason the module exists
User-facing summary	What layman sees (link to User Manual)
Depends on	Upstream modules / data
Used by	Downstream modules
Routes	Web route names / prefixes
Controller	Primary PHP class
Models	Eloquent models & key columns
Permissions	Spatie permission slugs
Module gate	CheckModuleAccess slug if any
Institution scoping	How to query safely
Key flows	Request lifecycle

Integration points	Notifications, jobs, API
Example	Concrete IDs / payloads
Pitfalls	Common bugs for developers

FOUNDATION

F1: Architecture & Multi-Tenancy

Purpose

Single Laravel application serves many schools. Data isolation uses `institution_id` on tenant tables and session context.

Core classes

Class	Role
<code>BaseController</code>	<code>getInstitutionId()</code> , <code>getAllowedInstitutionIds()</code> , <code>assertInstitutionMatch()</code> , <code>checkInstitution(Role)</code>
<code>LoadInstitutionSettings</code>	Middleware — loads settings into runtime
<code>CheckSubscription</code>	Blocks expired subscriptions (3-day grace)
<code>CheckModuleAccess</code>	Route middleware <code>CheckModuleAccess:module_slug</code>
<code>SetLocale</code>	EN/FR from session

`getInstitutionId()` resolution order

- Session `active_institution_id` if user allowed (or `'global'` → `null` for Super Admin)
- Fallback `users.institute_id`
- First ID from allowed list

Example

```
$institutionId = $this->getInstitutionId(); // 3 or null (global)

Student::where('institution_id', $institutionId)
    ->findOrFail($studentId);
```

Pitfalls

- **IDOR:** Never `Student::findOrFail($id)` without institution scope for school-scoped roles.
 - **Global context:** Super Admin with `null` institution must still validate when writing data.
-

F2: Authorization Stack

Layers

- **Route middleware:** `auth`, `verified`, `RoleMiddleware`, `PermissionMiddleware`
- **Spatie permissions:** `{singular_module}.{action}` e.g. `student.create`
- **Policies:** `ResourcePolicy` on models
- **Module access:** Package/subscription enabled modules

RolePermissionSeeder modules

See `database/seeder/RolePermissionSeeder.php` — `$modulesData` array defines all modules and actions.

Example permission check

```
if (!Auth::user()->can('invoice.view')) {  
    abort(403);  
}
```

F3: Notification Architecture

Services

Service	Responsibility
<code>NotificationPreferenceService</code>	<code>isChannelEnabled(\$institutionId, \$eventKey, \$channel)</code>
<code>NotificationService</code>	SMS, WhatsApp, email via gateways
<code>InAppNotificationService</code>	Rows in <code>in_app_notifications</code> , respects <code>system</code> channel

Settings key pattern

```
notify_{event_key} → JSON: {"sms":true,"whatsapp":false,"email":true,"system":true}
```

Stored in `institution_settings`.

Default policy

- No row saved → **system ON**, SMS/WhatsApp/email **OFF**
- Always-on events: `institution_created`, `low_balance`, `user_welcome`, etc.

Hook pattern in controllers

```
app(InAppNotificationService::class)->notifyPaymentReceived($payment);  
$this->notificationService->sendInvoiceNotification($invoice);
```

F4: Development Setup

```
composer install && npm install  
cp .env.example .env && php artisan key:generate  
# Set DB_*, HARDWARE_SECRET, APP_URL=https://e-digitex.com  
php artisan migrate  
php artisan db:seed --class=RolePermissionSeeder  
php artisan db:seed --class=SmsTemplateSeeder  
php artisan storage:link  
php artisan docs:generate-pdf    # regenerate manuals
```

Default admin: `admin@digitex.com` / `password`

MODULE REFERENCE (Alphabetical BY DOMAIN)

M01: Academic Sessions

Item	Value
Purpose	Define school years; one <code>is_current</code> drives defaults
Depends on	Institution
Used by	Enrollments, Fee structures, Exams, Invoices
Routes	<code>academic-sessions.*</code> resource
Controller	<code>AcademicSessionController</code>
Model	<code>AcademicSession</code> — <code>institution_id</code> , <code>name</code> , <code>start_date</code> , <code>end_date</code> , <code>is_current</code>
Module slug	<code>academic_sessions</code>
Permissions	<code>academic_session.viewAny</code> , <code>.create</code> , <code>.update</code> , <code>.delete</code>

Example

```
AcademicSession::where('institution_id', 3)
    ->where('is_current', true)
    ->first(); // 2025-2026
```

M02: Assignments

Item	Value
Purpose	Homework with deadlines
Depends on	Class sections, subjects, staff

Used by	Student portal API <code>GET /v1/student/homework</code> , chatbot
Routes	<code>assignments.*</code> , <code>assignments.get-subjects</code>
Controller	<code>AssignmentController</code>
Model	<code>Assignment</code>
Module slug	<code>assignments</code>

API exposure

`StudentPortalApiController::getHomework()` — 7-day window, enrolled class.

M03: Audit Logs

Item	Value
Purpose	Change history
Controller	<code>AuditLogController</code>
Model	<code>AuditLog</code>
Route middleware	Super Admin only
Trait	<code>LogsActivity</code> on key models

M04: Budgets & Fund Requests

Item	Value
Purpose	Department budgets and approval workflow
Depends on	Institution, finance permissions
Routes	<code>budgets.*</code> , <code>budgets.requests</code> , approve endpoints
Controller	<code>Finance\BudgetController</code>
Models	<code>Budget</code> , <code>BudgetCategory</code> , <code>FundRequest</code>

Permissions	<code>budget.approve_funds</code> for approval
Notifications	<code>notifyFundRequestSubmitted</code> , <code>notifyFundRequestProcessed</code>

Institution scoping (required)

```
FundRequest::where('institution_id', $institutionId)->findOrFail($id);
Budget::where('institution_id', $institutionId)->findOrFail($budgetId);
```

M05: Campuses

Item	Value
Model	<code>Campus</code> — <code>institution_id</code> , <code>name</code> , <code>address</code>
Controller	<code>CampusController</code>
Routes	<code>campuses.*</code>

M06: Chatbot

Item	Value
Purpose	WhatsApp/SMS conversational UI
Service	<code>ChatbotLogicService</code> (large state machine)
Web routes	<code>/chatbot/settings</code> , <code>keywords</code> , <code>sessions</code>
Controller	<code>ChatbotSettingController</code>
Models	<code>ChatbotKeyword</code> , <code>ChatSession</code>
API	POST <code>/api/v1/chatbot/webhook/{provider}</code>
Middleware	Role: Super Admin, Head Officer, School Admin on web routes

Webhook providers

`infobip` , `twilio` , `meta` , `mobishastra`

Credits

Messages debit institution `sms_credits` unless `chatbot_free_interactions` enabled.

M07: Class Sections

Item	Value
Model	<code>ClassSection</code> — <code>grade_level_id</code> , <code>staff_id</code> (homeroom), <code>institution_id</code>
Controller	<code>ClassSectionController</code>
Module slug	<code>class_sections</code>

Example

Grade 5 Section A → `id=45`, `grade_level_id=8`, `staff_id=22` (Mr. Dupont)

M08: Class Subjects

Item	Value
Purpose	Allocate subject + teacher to class
Controller	<code>ClassSubjectController</code>
Model	<code>ClassSubject</code>
Module slug	<code>class_subjects</code>

M09: Configuration

Item	Value
Controller	<code>ConfigurationController</code> , <code>SmsTemplateController</code>
Routes	<code>/configuration/*</code> under role middleware

Storage	<code>institution_settings</code> key/value groups: <code>smtp</code> , <code>sms</code> , <code>notifications</code> , <code>academics</code> , <code>system</code>
----------------	--

SMS credential keys (examples)

Key	Encrypted?
<code>sms_provider</code>	No — e.g. <code>twilio</code>
<code>whatsapp_provider</code>	No — e.g. <code>meta</code>
<code>twilio_sid</code> , <code>twilio_from</code>	No
<code>twilio_token</code>	Yes
<code>meta_access_token</code>	Yes
<code>infobip_api_key</code>	Yes
<code>mobishastra_password</code>	Yes

updateSms flow

- Validate providers selected
- Save public keys loop
- Encrypt secrets via `Crypt::encryptString`
- Super Admin in **Global View** (`getInstitutionId()` = null) saves:
 - `sms_provider` / `whatsapp_provider` globally → overrides `config/sms.php` / `.env`
 - Allowed provider lists (`allowed_sms_providers` , `allowed_whatsapp_providers`)
 - Global API credentials (`institution_id` = null)
- `SystemCommunicationConfigService::applyGlobalOverrides()` runs on boot and after global save

Provider resolution (school sends message)

School setting	Provider used	Credentials from
----------------	---------------	------------------

<code>sms_provider = system</code>	Global <code>sms_provider</code> or <code>config('sms.default')</code>	Global settings (null institution)
<code>sms_provider = twilio</code>	Twilio	That school's settings

Helper: `InstitutionSetting::resolveSystemProvider('sms' | 'whatsapp')`

Global View requirement

Super Admin must set `session('active_institution_id')` to `'global'` (via header **Global View**) before opening Configuration → ID Sender SMS. Otherwise settings apply to the currently selected school only.

Test endpoints

- `configuration.smtp.test`
- `configuration.sms.test` — uses `NotificationService::performSend` with unlimited flag

M10: Dashboard

Item	Value
Controller	<code>DashboardController</code>
Views	<code>dashboard/super_admin</code> , <code>main_admin</code> , <code>teacher</code> , <code>student</code> , <code>accountant</code> , etc.
Route	<code>dashboard</code>

Role detected via `Auth::user()->hasRole()` and redirects to appropriate view with real stats queries scoped by `$institutionId`.

M11: Departments

Item	Value
Model	<code>Department</code>

Controller	DepartmentController
Module slug	departments
Used by	Subjects, Programs (university)

M12: Elections & Voting

Item	Value
Controllers	ElectionController, StudentVotingController, VotingController
Models	Election, ElectionPosition, Candidate, Vote
Module slug	elections
Policy	ElectionPolicy via authorizeResource

Security note

ElectionController DataTables `orderColumn` must whitelist sort direction (ASC/DESC only).

M13: Exams, Schedules, Marks, Results

Component	Controller	Model
Exams	ExamController	Exam — status : draft → published
Schedules	ExamScheduleController	ExamSchedule
Marks	ExamMarkController	ExamRecord
Result cards	ResultCardController	—
Reports	ReportController	bulletin, transcript views

Publish flow

ExamController::finalize → sets published → InAppNotificationService::notifyExamPublished

Student API

`StudentPortalApiController::getResults()` — only `exam.status = published`, debt block check.

Hardware API

`AttendanceApiController::handleReportCard()` — same published filter.

M14: Fee Types & Fee Structures

Item	Value
Controllers	<code>FeeTypeController</code> , <code>FeeStructureController</code>
Models	<code>FeeType</code> , <code>FeeStructure</code> , <code>FeeStructureItem</code>
Module slugs	<code>fee_types</code> , <code>fee_structures</code>

Example structure

Grade 5 Tuition 2025-2026 — 3 tranches × 150,000 XOF linked to `grade_level_id=8`.

M15: Global Search

Item	Value
Controller	<code>GlobalSearchController</code>
Service	<code>GlobalSearchService</code>
Route	<code>global-search.suggest?q=</code>
Auth	Permission-scoped results per entity type

M16: Grade Levels

Item	Value
------	-------

Model	<code>GradeLevel</code> — <code>name</code> , <code>order</code> , <code>institution_id</code>
Controller	<code>GradeLevelController</code>
Module slug	<code>grade_levels</code>

M17: Head Officers

Item	Value
Controller	<code>HeadOfficersController</code>
Middleware	Super Admin only
Pivot	<code>institution_head_officers</code>
User type	<code>UserType::HEAD_OFFICER</code>

M18: In-App Notifications

Item	Value
Model	<code>InAppNotification</code>
Service	<code>InAppNotificationService</code>
Controller	<code>InAppNotificationController</code>
Composer	<code>AppServiceProvider</code> shares <code>\$inAppNotifications</code> to header
Routes	<code>notifications.read</code> , <code>notifications.read_all</code>

Event key map

See `InAppNotificationService::EVENT_KEYS` — maps internal type to `notify_*` preference key.

M19: Institutions

Item	Value
------	-------

Model	Institution
Controller	InstituteController
Relations	campuses, subscriptions, settings

M20: Invoices & Payments

Component	Controller	Key logic
Invoices	Finance\InvoiceController	Bulk generate, PDF print
Payments	Finance\PaymentController	Links to invoice, triggers notifications

Scoping pattern

```
Invoice::when($institutionId, fn($q) => $q->where('institution_id', $institutionId))
->findOrFail($id);
```

Notifications

- notifyInvoiceCreated
- notifyPaymentReceived
- NotificationService::sendInvoiceNotification

M21: Modules & Permissions (System)

Item	Value
Models	Module, Spatie Permission, Role
Controllers	ModuleController, PermissionController, RolesController, RolePermissionController
PermissionController	Super Admin only for mutations
RolePermissionController	Requires checkInstitution(\$role) + role.update

M22: Notices & Reminders

Item	Value
Notices	NoticeController , StudentNoticeController
Model	Notice — audience , is_published
Reminders	ReminderController — fee/exam bulk SMS
Module slug	notices

Notice notification

notifyNoticePublished — audience maps to role lists.

M23: Packages & Subscriptions

Item	Value
Controller	SubscriptionController
Models	Package , Subscription , PlatformInvoice
Middleware	Super Admin / Head Officer for platform billing

Package.modules JSON → drives CheckModuleAccess .

M24: Parents

Item	Value
Model	StudentParent
Controller	ParentController
Relation	Student.parent_id , optional user_id for portal

M25: Payroll & Salary Structures

Item	Value
Controllers	<code>PayrollController</code> , <code>SalaryStructureController</code>
Models	<code>Payroll</code> , <code>SalaryStructure</code> , <code>PayrollItem</code>
Module slugs	<code>payrolls</code> , <code>salary_structures</code>

M26: Pickup (Web + API)

Surface	Controller
Web guard/teacher/parent	<code>PickupWebController</code>
Mobile staff	<code>Api\V1\AppPickupController</code>
Hardware scan	<code>Api\V1\AttendanceApiController::handlePickup</code>
Chatbot QR	<code>Api\V1\Chatbot\PickupController</code>

Model	<code>StudentPickup</code> — statuses: pending → scanned → approved/rejected
-------	--

Authorization

- Web `updateStatus` — teacher/admin roles + institution scope
- API OTP — guardian/staff + `userCanAccessStudent`
- API approve — institution membership for admins

M27: Programs & Academic Units (LMD)

Item	Value
Controllers	<code>ProgramController</code> , <code>AcademicUnitController</code>
Models	<code>Program</code> , <code>AcademicUnit</code>
Routes	<code>programs.*</code> , <code>units.*</code>

Program store fix pattern

Update only within institution:

```
Program::where('institution_id', $institutionId)->findOrFail($id);
```

M28: Settings

Item	Value
Controller	SettingsController
Storage	institution_settings keys: attendance_locked, exams_locked, grading_scale, block_reports_on_debt, active_periods
Permission	setting.manage or admin roles

M29: SMS Templates

Item	Value
Model	SmsTemplate — event_key, body, available_tags
Seeder	SmsTemplateSeeder — global templates
Controller	SmsTemplateController via configuration

Institution can override template per event_key .

M30: Staff, Staff Attendance, Staff Leave

Module	Controller	Model
Staff	StaffController	Staff → User
Staff attendance	StaffAttendanceController	StaffAttendance
Staff leave	StaffLeaveController	StaffLeave

Staff leave scoping

All find/update operations must use `where('institution_id', $institutionId)`.

M31: Student Attendance

Item	Value
Web	<code>StudentAttendanceController</code> , <code>AttendanceReportController</code>
Hardware	<code>AttendanceApiController</code>
Model	<code>StudentAttendance</code> — <code>check_in</code> , <code>check_out</code> , <code>status</code> , <code>institution_id</code>

Hardware UID lookup (correct pattern)

```
Student::where(function ($q) use ($uids) {
    $q->whereIn('nfc_tag_uid', $uids)
    ->orWhereIn('rfid_uid', $uids)
    // ...
})->first();
```

Never ungrouped `whereIn()` -> `orWhereIn()` on tenant tables.

Hardware auth

`HARDWARE_SECRET` required; `hash_equals`; `getTodayScans` requires `X-Institution-Id` for hardware mode.

M32: Student Enrollments & Promotion & Transfers

Module	Controller
Enrollments	<code>StudentEnrollmentController</code>
University	<code>UniversityEnrollmentController</code>
Promotion	<code>StudentPromotionController</code>
Transfers	<code>TransferController</code>

M33: Student Requests

Item	Value
Controller	StudentRequestController
Model	StudentRequest
Notifications	submit + status update
Destroy	Institution scope + owner/admin check

Teachers blocked from index (query `whereRaw('1=0')`).

M34: Students

Item	Value
Controller	StudentController
Model	Student — hardware fields, <code>user_id</code> , <code>parent_id</code>
Module slug	students

Central hub model — most modules reference `student_id` .

M35: Subjects

Item	Value
Controller	SubjectController
Model	Subject
University	May link to department / academic unit

M36: Timetables

Item	Value
Controller	TimetableController
Model	Timetable — day_of_week, start_time, class_section_id, teacher_id
Pitfall	Do not mass-assign institution_id from \$request->all() on update

M37: User Profile

Item	Value
Web	UserProfileController
API	UserProfileApiController
Model	User — institute_id, fcm_token, language

API MODULE MAP (Summary)

Prefix	Controller	Auth
/v1/login	AuthApiController	Public
/v1/profile/*	UserProfileApiController	Sanctum
/v1/hardware/*	AttendanceApiController	Hardware secret
/v1/pickup/*	AppPickupController	Sanctum
/v1/student/*	StudentPortalApiController	Sanctum
/v1/chatbot/*	Chatbot controllers	Webhook public / QR Sanctum

Full request/response examples: **REST API Manual** PDF.

ADDING A NEW MODULE — CHECKLIST

- Migration with `institution_id` FK indexed
 - Model + `$fillable` without unsafe fields
 - `ResourcePolicy` + register
 - Entry in `RolePermissionSeeder` `$modulesData`
 - `Module` record slug matches route middleware
 - Controller extends `BaseController`, scopes queries
 - Routes inside `CheckModuleAccess:slug` group
 - Lang files `resources/lang/en/{domain}.php` + `fr`
 - Sidebar entry (if not auto-discovered)
 - Optional: `SmsTemplateSeeder` event + `InAppNotificationService` method
 - Feature tests for institution isolation
-

ENVIRONMENT & DEPLOYMENT

Variable	Module impact
HARDWARE_SECRET	M31 hardware attendance
APP_URL	Must be <code>https://e-digitex.com</code> in production — webhooks, QR links, asset URLs (single URL for all schools)
SANCTUM_STATEFUL_DOMAINS	SPA auth
SMS env fallbacks	M09 when DB provider = <code>system</code>
<code>storage/app/firebase-credentials.json</code>	FCM push on pickup approve

Production commands

```
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan queue:work
```

Production URL (multi-tenant)

- Set `APP_URL=https://e-digitex.com` (no trailing path).
 - **All institutions share one login URL:** `https://e-digitex.com/login`
 - Tenant context comes from the user account (`institute_id` , head-officer pivot) and session `active_institution_id` — not from DNS subdomains.
 - Chatbot webhooks: `https://e-digitex.com/api/v1/chatbot/webhook/{provider}`
 - Mobile app and hardware API base: `https://e-digitex.com/api/v1/...`
-

TESTING RECOMMENDATIONS

Area	Test
Finance	User from institution A cannot view invoice from B
Pickup	Guardian cannot OTP unrelated student
Hardware	Missing secret returns 503
Roles	Non-admin cannot assign permissions
Notifications	Disabled system channel skips InAppNotification row

MODULE DEPENDENCY GRAPH (Developer View)

Institution

- |— InstitutionSetting (config, modules, notify_*)
- |— Subscription → Package.modules
- |— AcademicSession (is_current)
 - | |— FeeStructure → Invoice → Payment
 - | |— Exam → ExamRecord
 - | |— StudentEnrollment → ClassSection → GradeLevel
- |— Student (+ StudentParent)
 - | |— StudentAttendance ← hardware UID
 - | |— StudentPickup
 - | |— StudentRequest
- |— Staff (+ User + Role)
 - | |— Timetable.teacher_id
 - | |— StaffLeave
 - | |— Payroll

*End of Developer Manual — Regenerate PDF: `php artisan docs:generate-pdf` *